LANGGRAPH TO WRITE PLAYWRIGHT SCRIPT

AI-Based Playwright Test Script Generator Using LangGraph and Google Gemini

2. Purpose of the Project

The purpose of this project is to automate the generation of Playwright test scripts using Artificial Intelligence. The system uses a Large Language Model (LLM) to generate complete JavaScript-based Playwright automation scripts based on user-defined testing requirements.
This project aims to reduce manual effort in writing automation test cases, improve productivity of QA engineers, and demonstrate the integration of AI with test automation frameworks.

---

3. Project Description

This project is a Python-based AI automation tool that generates Playwright test scripts automatically. The system uses LangGraph for workflow management, LangChain for LLM communication, and Google Gemini API for generating intelligent automation code.
The application sends a predefined automation testing prompt to the AI agent. The AI generates a complete Playwright JavaScript test script. The system then extracts the generated JavaScript code using regular expressions and saves it as a .spec.js file on the local system.

This project is useful for automation testers, QA engineers, and students who want to quickly generate test scripts for web applications.

---

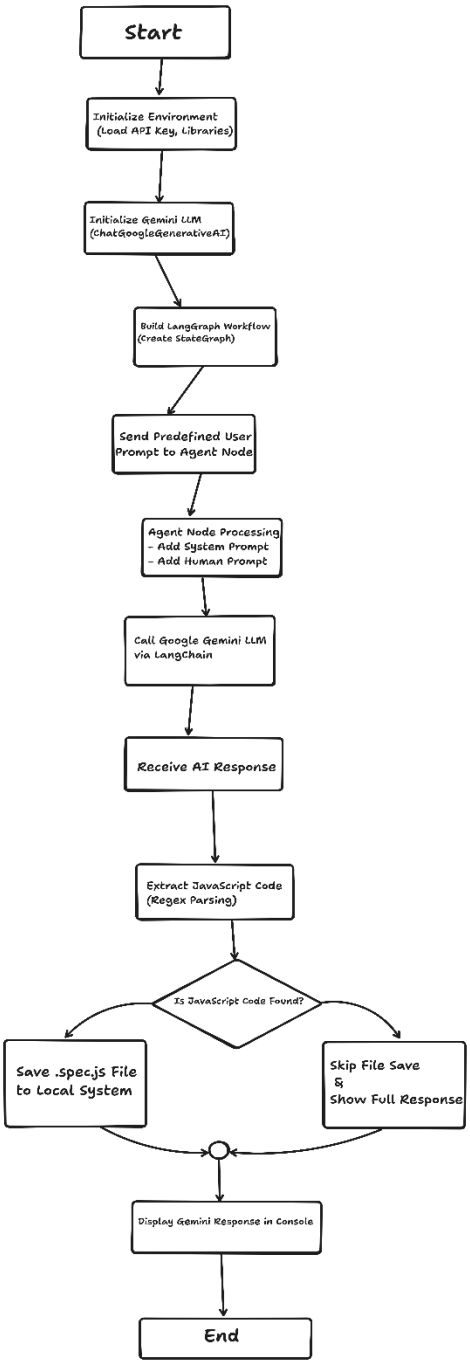
4. Technology Stack

Programming Language: Python
Test Automation Framework: Playwright (JavaScript)
AI Model: Google Gemini (gemini-3-flash-preview)
Frameworks/Libraries:
LangGraph
LangChain
langchain_google_genai
Scripting Language for Tests: JavaScript
Tools:
VS Code
Node.js (for running Playwright tests)
Platform: Windows

---

4. WORKFLOW

System initializes environment variables and required libraries.
Google Gemini LLM is initialized using LangChain.
LangGraph workflow is created with an agent node.
A predefined automation testing prompt is sent to the agent node.
Agent node adds system and user messages.
The prompt is sent to the Gemini LLM.
The LLM generates Playwright JavaScript test code.
The system parses the AI response using regular expressions.
If JavaScript code is found, it is saved as a .spec.js file.
The full AI response is displayed in the console.



6. System Design

6.1 ARCHITECTURE (TEXTUAL REPRESENTATION)

User/Predefined Prompt
→ Python Application
→ LangGraph State Manager
→ LangChain Interface
→ Google Gemini LLM API
→ AI-Generated Playwright Script
→ Regex-Based Code Extraction
→ Save .spec.js File
→ Console Output

6.2 MAJOR MODULES

Environment Setup Module: Loads API key and required libraries.
Agent Module: Prepares system and user prompts for LLM.
LLM Communication Module: Sends and receives data from Gemini API.
Workflow Management Module: Manages execution using LangGraph.
Code Extraction Module: Extracts JavaScript code using regex.
File Handling Module: Saves generated Playwright script to local file.
Output Module: Displays AI response in console.

6.3 DATA FLOW

Input prompt is passed to the LangGraph agent node.
Agent node forwards request to Gemini LLM.
Gemini LLM returns AI-generated response.
System parses response to extract JavaScript code.
Extracted code is written to a .spec.js file.
Final response is printed to the console.